Manual Técnico — Sistema de Gestión Documental

Índice

1. Introducción
2. Arquitectura del sistema
3. Tecnologías utilizadas
4. Estructura del repositorio
5. Backend (NestJS)
6. Frontend (Angular)
7. Base de datos (MongoDB)
8. Autenticación y autorización
9. Instalación y ejecución
10. Carga inicial de datos (seed)
11. Decisiones de diseño
12. Mantenimiento y extensión

1. Introducción

El Sistema de Gestión Documental es una aplicación cliente-servidor que permite almacenar, organizar, consultar y compartir documentos digitales (.txt y .md) entre usuarios de una organización.
El sistema está compuesto por dos aplicaciones independientes:

- Una API REST desarrollada con NestJS (Node.js) que gestiona la lógica de negocio, la autenticación y el acceso a la base de datos.
- Una SPA (Single Page Application) desarrollada con Angular que consume la API y ofrece la interfaz de usuario.

La base de datos es MongoDB, que almacena los metadatos en colecciones y el contenido de los archivos en GridFS.
El objetivo técnico del proyecto es demostrar el uso de una arquitectura cliente-servidor con una base de datos NoSQL orientada a documentos, aplicando decisiones de modelado (embebido vs. referencia), índices, agregaciones y control de acceso basado en roles.

2. Arquitectura del sistema

2.1 Vista general

El sistema tiene tres capas principales:

1. Cliente (navegador): SPA Angular que se comunica con la API mediante HTTP/JSON.
2. Servidor de aplicaciones: API NestJS que expone endpoints REST bajo el prefijo /api.
3. Servidor de base de datos: MongoDB, que puede correr en Docker o en local.

Además, hay un contenedor auxiliar (mongo-express) para inspeccionar la base de datos desde el navegador.

2.2 Flujo de una petición

1. El usuario interactúa con la SPA.
2. La SPA envía una petición HTTP a la API, con el token JWT en la cabecera Authorization.
3. El JwtAuthGuard valida el token y busca al usuario en la base de datos.
4. El RolesGuard comprueba que el rol del usuario tenga permiso para el endpoint.
5. El ValidationPipe valida el cuerpo de la petición contra el DTO.
6. El controller delega en el service.
7. El service aplica la lógica de negocio y llama a los repositorios.
8. Los repositorios consultan MongoDB (con Mongoose) o GridFS.
9. La respuesta viaja de vuelta al cliente como JSON.

2.3 Componentes desplegables

- Cliente: SPA Angular, servida en el puerto 4200 (en desarrollo) o como build estático en producción.
- Servidor de aplicaciones: API NestJS, en el puerto 3000 (configurable por la variable PORT).
- MongoDB: puerto 27017 dentro del contenedor, mapeado al 27018 en el host (para evitar conflicto con un MongoDB global).
- mongo-express: puerto 8081.

3. Tecnologías utilizadas

3.1 Backend

- Node.js 20+ (probado con 24.21.0).
- NestJS 12: framework modular para Node.js.
- Mongoose 9: ODM para MongoDB.
- @nestjs/mongoose: integración de Mongoose con NestJS.
- @nestjs/jwt: generación y verificación de JWT.
- @nestjs/config: carga de variables de entorno.
- @nestjs/platform-express: integración con Express (Multer, CORS).
- bcryptjs: hashing de contraseñas.
- class-validator y class-transformer: validación y transformación de DTOs.
- Multer: manejo de subida de archivos (multipart/form-data).
- GridFS: almacenamiento de binarios en MongoDB.

3.2 Frontend

- Angular 21 LTS: framework SPA.
- Angular Material: componentes de UI.
- RxJS: programación reactiva (peticiones HTTP).
- Signals y computed: estado reactivo moderno de Angular.
- Reactive Forms: formularios con validación.
- HttpInterceptor: inyección del token JWT y manejo de 401.

- Guards de ruta: authGuard (autenticación) y rolGuard (roles).

3.3 Base de datos

- MongoDB 7 (contenedor Docker mongo:7).
- GridFS: almacenamiento de contenido binario en dos colecciones (contenidos.files y contenidos.chunks).
- Índices: simples, compuestos, únicos, parciales y con collation.

3.4 Infraestructura

- Docker y Docker Compose: MongoDB y mongo-express.
- Git y GitHub: control de versiones y entrega.

4. Estructura del repositorio

Proyecto2_Archivos/

- docker-compose.yml: MongoDB + mongo-express.
- README.md: instrucciones de arranque.
- docs/: documentación (modelo documental, manuales, diagramas UML).
- gestion-documental-backend/: API NestJS.
    - src/: código fuente.
    - .env.example: plantilla de variables de entorno.
    - package.json: dependencias y scripts.
- gestion-documental-frontend/: SPA Angular.
    - src/: código fuente.
    - package.json: dependencias y scripts.

4.1 Estructura del backend

gestion-documental-backend/src/

- main.ts: punto de entrada (CORS, prefijo /api, ValidationPipe, puerto).
- app.module.ts: módulo raíz (ConfigModule, MongooseModule, módulos funcionales).
- autenticacion/: login, registro, JWT, guards, decoradores.
- usuarios/: CRUD de usuarios, roles, activar/desactivar.
- carpetas/: CRUD de carpetas, mover, renombrar, jerarquía.
- archivos/: subida, edición, mover, renombrar, contenido (GridFS).
- comparticiones/: compartir, cambiar nivel, revocar.
- comentarios/: crear, listar, eliminar.
- papelera/: enviar, restaurar, eliminar definitivamente, vaciar.
- acciones/: historial de actividad (registro y consulta).
- reportes/: 12 reportes con agregaciones.
- acceso/: resolución de permisos (propietario, lectura, comentar, edición).
- common/: enums, pipes, utilidades, constantes.
- database/seed.ts: script de carga inicial.

Cada módulo sigue el patrón:

- controller: expone los endpoints.
- service: lógica de negocio.

- repository: acceso a MongoDB.
- entities/: schema de Mongoose.
- dto/: validación de entrada.
- interfaces/: tipos internos.

4.2 Estructura del frontend

gestion-documental-frontend/src/app/

- auntenticacion/: login, registro, AuthService, guards.
- carpetas/: explorador de carpetas.
- archivos/: visor y editor de archivos.
- comparticiones/: recursos compartidos y recibidos.
- comentarios/: panel de comentarios.
- papelera/: gestión de la papelera.
- usuarios/: administración de usuarios (admin).
- reportes/: pantalla de reportes y componentes por reporte.
- interfaz/: sidebar, layout.
- core/: ApiService, guards, interceptores, modelos.
- shared/: pipes y componentes reutilizables.

5. Backend (NestJS)

5.1 Punto de entrada

El archivo main.ts configura:

- Prefijo global /api para todos los endpoints.
- CORS con el origen configurado en CORS_ORIGIN.
- ValidationPipe global con whitelist, forbidNonWhitelisted y transform.
- Puerto configurable por PORT.

5.2 Módulo raíz

El app.module.ts importa:

- ConfigModule.forRoot({ isGlobal: true }): carga el .env.
- MongooseModule.forRootAsync: conecta a MongoDB con MONGODB_URI.
- Los módulos funcionales (usuarios, autenticación, carpetas, archivos, etc.).

5.3 Patrón controller-service-repository

Cada módulo funcional sigue el mismo patrón:

- Controller: recibe la petición, valida el DTO, delega en el service.
- Service: aplica la lógica de negocio, coordina repositorios, lanza excepciones HTTP.
- Repository: encapsula las consultas a MongoDB con Mongoose.

Esto separa responsabilidades y facilita el testing.

5.4 Manejo de errores

Los servicios lanzan excepciones de NestJS:

- BadRequestException (400): datos inválidos.
- UnauthorizedException (401): token ausente o inválido.
- ForbiddenException (403): sin permisos.
- NotFoundException (404): recurso no encontrado.
- ConflictException (409): duplicado (por índice único).

Los errores de MongoDB (código 11000) se detectan con la utilidad esClaveDuplicada y se transforman en ConflictException.

5.5 Validación de entrada

Los DTOs usan class-validator:

- @IsMongoId, @IsEmail, @IsEnum, @IsString, @IsInt.
- @Min, @Max, @MaxLength, @MinLength.
- @IsOptional, @ValidateIf.
- @Transform(recortarTexto) para normalizar.
- @Type(() => Date) y @Type(() => Number) para castear.

El ValidationPipe global aplica estos DTOs antes de que la petición llegue al controller.

5.6 Subida de archivos

La subida usa Multer (FileInterceptor) con un límite configurable por ARCHIVO_MAX_MB. El archivo se recibe como buffer, se valida (UTF-8, extensión) y se guarda en GridFS. Si falla la creación del documento, se elimina el contenido de GridFS para no dejar huérfanos.

5.7 Almacenamiento en GridFS

El ContenidosRepository usa GridFSBucket con el nombre de bucket "contenidos". Genera dos colecciones:

- contenidos.files: metadatos (nombre, tamaño, chunkSize, uploadDate).
- contenidos.chunks: trozos binarios de 255 KB.

El documento de archivos solo guarda el contenidoId (referencia al _id de contenidos.files).

6. Frontend (Angular)

6.1 Estructura general

La SPA usa componentes standalone (sin NgModule), con imports explícitos en cada componente. El estado se gestiona con signals y computed.

6.2 Autenticación

El AuthService gestiona:

- Login: POST /api/auth/login, guarda el token y el usuario en localStorage.
- Registro: POST /api/auth/registro.
- Logout: limpia el localStorage y redirige a /login.
- Estado reactivo: signals para usuario, token, estaAutenticado, esAdmin.

El authInterceptor añade el token JWT en la cabecera Authorization de cada petición a la API (solo a environment.apiUrl) y, si recibe un 401 (excepto en login), cierra la sesión automáticamente.

6.3 Guards de ruta

- authGuard: protege las rutas que requieren autenticación.
- rolGuard: protege las rutas que requieren un rol concreto (por ejemplo, /usuarios para ADMIN).

6.4 Comunicación con la API

Los servicios extienden ApiService, que expone http (HttpClient) y baseUrl (environment.apiUrl). Cada servicio define sus métodos (listar, crear, actualizar, eliminar) y devuelve Observables.

6.5 Componentes principales

- Login y Registro: formularios reactivos con validación.
- Explorador de carpetas: listado, creación, navegación.
- Visor de archivos: vista y edición de contenido.
- Compartidos: recursos recibidos y otorgados.
- Papelera: listado, restauración y eliminación.
- Reportes: 12 reportes con tablas.
- Usuarios: administración (solo admin).

6.6 Manejo de errores en el cliente

Cada componente captura errores HTTP y muestra mensajes al usuario con MatSnackBar o mensajes inline. Los errores 401 redirigen a /login a través del interceptor.

7. Base de datos (MongoDB)

7.1 Colecciones

- usuarios: cuentas (admin y usuario).
- carpetas: jerarquía de carpetas.
- archivos: metadata de archivos (sin contenido).
- comparticiones: permisos de recursos a usuarios.
- comentarios: comentarios sobre archivos.
- actividades: historial de operaciones.
- contenidos.files y contenidos.chunks: GridFS.

La descripción completa del modelo (embebido vs. referencia, índices, decisiones) está en el documento docs/modelo-documental.md.

7.2 Índices

Los índices principales son:

- usuarios.correo: único.
- carpetas { propietario, carpetaPadre, nombre }: único parcial, con collation es.
- carpetas.ancestros: simple.
- archivos { propietario, carpeta, nombre }: único parcial, con collation es.
- archivos { propietario, createdAt }: compuesto.
- archivos.etiquetas: simple.
- comparticiones { recurso, destinatario }: único.
- comparticiones { destinatario, createdAt }: compuesto.
- comparticiones { propietario, createdAt }: compuesto.
- comentarios { archivo, createdAt }: compuesto.
- actividades { usuario, fecha }, { recurso, fecha }, { fecha }, { propietarioRecurso, fecha }: compuestos/simples.

7.3 Operaciones destacadas

- Update pipeline en CarpetasRepository.moverConDescendientes: reescribe ancestros de toda la descendencia en una sola operación.
- $inc en UsuariosRepository.ajustarAlmacenamiento: ajuste atómico del almacenamiento usado.
- $lookup con pipeline en reportes: enriquecer datos con filtros y proyecciones.
- $facet en reportes paginados: obtener datos y total en una sola consulta.

---

8. Autenticación y autorización

8.1 JWT

- El login valida las credenciales con bcrypt.compare.
- Si son válidas, firma un JWT con el payload { sub: usuario.id, rol: usuario.rol }.
- El token expira según JWT_EXPIRA_SEGUNDOS (por defecto, 28800 segundos = 8 horas).
- El token se firma con JWT_SECRET (variable de entorno).

8.2 Guards

- JwtAuthGuard: extrae el token de la cabecera Authorization (Bearer), lo verifica con JwtService.verifyAsync, busca al usuario en la base de datos y, si está activo, lo pone en la request.
- RolesGuard: comprueba que el rol del usuario esté entre los permitidos por el decorador @Roles.

Ambos están registrados como APP_GUARD, por lo que se aplican a toda la aplicación. El decorador @Publico exime una ruta de la autenticación.

8.3 Roles

- ADMIN: acceso total, incluidos reportes globales y administración de usuarios.
- USUARIO: acceso a sus propios recursos y a los compartidos con él, más los reportes "por usuario".

8.4 Permisos sobre recursos

El AccesoService resuelve el nivel de acceso de un usuario sobre un archivo o carpeta:

- Si es el propietario: PROPIETARIO.
- Si no, busca comparticiones directas o en la cadena de ancestros de la carpeta: LECTURA, COMENTAR o EDICION.
- El nivel más alto encontrado es el que se aplica.

Los servicios que requieren un nivel mínimo llaman a exigirArchivo o exigirCarpeta, que lanzan ForbiddenException si no se cumple.

---

9. Instalación y ejecución

9.1 Requisitos

- Node.js 20+.
- Docker y Docker Compose.
- Angular CLI (opcional, para el frontend).

9.2 Pasos

1. Levantar MongoDB: docker compose up -d (desde la raíz).
2. Backend:
    - cd gestion-documental-backend
    - npm install
    - cp .env.example .env
    - npm run seed (solo si la base está vacía)
    - npm run start:dev
3. Frontend:
    - cd gestion-documental-frontend
    - npm install
    - ng serve

9.3 Variables de entorno del backend

- PORT: puerto del backend (por defecto 3000).
- MONGODB_URI: cadena de conexión a MongoDB.
- CORS_ORIGIN: origen permitido (por defecto http://localhost:4200).
- JWT_SECRET: secreto para firmar los JWT.
- JWT_EXPIRA_SEGUNDOS: expiración del token en segundos.
- ARCHIVO_MAX_MB: tamaño máximo de archivo en MB.

El .env.example contiene la plantilla con valores por defecto para desarrollo.

---

10. Carga inicial de datos (seed)

El script src/database/seed.ts crea:

- 7 usuarios (1 admin, 6 usuarios; uno de ellos inactivo).

- Estructura de carpetas por usuario (hasta 3 niveles).
- Archivos .txt y .md con contenido generado (60 archivos).
- Comparticiones entre usuarios (al menos 20).
- Comentarios en archivos.
- Actividades de todo tipo (subidas, descargas, ediciones, etc.).
- Recursos en la papelera (con cascada).
- Fechas repartidas en los últimos 60 días.

Se ejecuta con npm run seed (solo si la base está vacía) o npm run seed:reset (borra y resiembra, incluyendo GridFS).
El seed no requiere que el backend esté corriendo: usa NestFactory.createApplicationContext para abrir un contexto aislado, sembrar y cerrarse.

Glosario
1. Operadores de agregación ($)

$match
Filtra documentos antes de agrupar o transformar. Equivale a un WHERE en SQL.
Dónde se usa: en la mayoría de los aggregate de ReportesRepository ({ $match: { enPapelera: false } }, { $match: { tipoRecurso: 'ARCHIVO' } }), y dentro del pipeline del $lookup para filtrar el recurso relacionado.
Por qué: reduce el conjunto de datos antes de agrupar; aprovecha los índices y acelera la consulta.

$group
Agrupa documentos por una clave (_id) y aplica acumuladores ($sum, $avg, $push, etc.).
Dónde se usa: en todos los reportes de agrupación, por ejemplo { $group: { _id: '$accion', total: { $sum: 1 } } } en operacionesPorTipo.
Por qué: es el corazón de los reportes; agrupa por la clave del informe y cuenta o suma.

$sort
Ordena los documentos. 1 = ascendente, -1 = descendente.
Dónde se usa: en los "top N" de los reportes ({ $sort: { total: -1 } }) y en el historial (fecha: -1).
Por qué: los tops van de mayor a menor; el historial, de más reciente a más antiguo.

$limit
Corta el pipeline después de N documentos.
Dónde se usa: en todos los "top N" de reportes, después del $sort.
Por qué: un top 10 no necesita calcular más de 10.

$skip
Salta N documentos. Se usa para paginación junto con $limit.
Dónde se usa: en ArchivosRepository.buscar (.skip((pagina - 1) * limite)) y en archivosNuncaCompartidos ($facet con $skip).
Por qué: paginación clásica.

$lookup
Hace un "join" con otra colección. Trae documentos relacionados.
Dónde se usa: en los reportes que necesitan datos de dos colecciones (archivos, carpetas, usuarios).
Por qué: los reportes cruzan información; el $lookup es el JOIN de MongoDB.

$lookup con pipeline
Forma avanzada del $lookup: permite filtrar y proyectar dentro del join.
Dónde se usa: en la mayoría de reportes, por ejemplo pipeline: [{ $match: { enPapelera: false } }, { $project: { _id: 1, nombre: 1 } }].
Por qué: sin el pipeline, el $lookup trae el documento completo (incluido password en usuarios); con él, se filtra y se proyecta solo lo necesario.

$lookup con let y $expr
Permite usar variables locales en el pipeline del $lookup y condiciones compuestas.
Dónde se usa: en archivosNuncaCompartidos (let: { archivoId: '$_id' } y $expr: { $eq: ['$recurso', '$$archivoId'] }).
Por qué: para comparar el campo de un documento con una variable; imprescindible cuando el join necesita más de una condición.

$unwind
Deshace un array, generando un documento por cada elemento.
Dónde se usa: después de los $lookup que devuelven un array de 1 elemento, para convertirlo en objeto ({ $unwind: '$archivo' }).
Por qué: $lookup siempre devuelve array; $unwind lo convierte en objeto y descarta los documentos cuyo array está vacío.

$project
Selecciona, renombra o excluye campos. 1 incluye, 0 excluye; también permite renombrar ({ compartidoEn: '$createdAt' }).
Dónde se usa: en los $lookup con pipeline ({ $project: { _id: 1, nombre: 1, correo: 1 } }) y en comparticionesOtorgadas/Recibidas (renombrar createdAt a compartidoEn).
Por qué: no traer más datos de los necesarios y renombrar para el frontend.

$facet
Ejecuta varios pipelines en paralelo sobre el mismo conjunto de datos.
Dónde se usa: en archivosNuncaCompartidos ($facet: { datos: [skip, limit], total: [count] }).
Por qué: obtiene la página y el total en una sola consulta, sin dos viajes a MongoDB.

$count
Cuenta documentos y los mete en un campo con el nombre indicado.
Dónde se usa: en archivosNuncaCompartidos ({ $count: 'n' } dentro del $facet).
Por qué: para el total paginado.

$size
Devuelve el tamaño de un array.
Dónde se usa: en archivosNuncaCompartidos ({ $match: { comparticiones: { $size: 0 } } }).
Por qué: $size: 0 equivale a "array vacío", es decir, "sin comparticiones".

$ne
"Distinto de".
Dónde se usa: en carpetasConMasArchivos ({ carpeta: { $ne: null } }) y en buscarEnDirectorio ({ _id: { $ne: excluirId } }).
Por qué: descartar la raíz o excluir al propio usuario.

$gte y $lte
"Mayor o igual que" y "menor o igual que".
Dónde se usa: en los filtros por rango de fechas (ArchivosRepository.buscar, ActividadesRepository.buscar, archivosPorFechas).
Por qué: rangos inclusivos; $gt/$lt serían exclusivos.

$in
"Está dentro del array".
Dónde se usa: en CarpetasRepository.nombresDe, ArchivosRepository.buscarPorCarpetas, ComparticionesRepository.eliminarPorRecursos, AccesoRepository.permisos.
Por qué: batch de operaciones; evita N consultas.

$or
"Se cumple A o B".
Dónde se usa: en CarpetasRepository.moverConDescendientes, ActividadesRepository.buscar, UsuariosRepository.buscarEnDirectorio.
Por qué: condiciones alternativas (buscar en varios campos o afectar varios documentos).

$expr, $eq, $and
$expr permite usar expresiones de agregación en un filtro; $eq es igualdad; $and es conjunción.
Dónde se usa: en archivosNuncaCompartidos ($expr: { $and: [{ $eq: [...] }, { $eq: [...] }] }).
Por qué: comparar campos entre documentos (no con literales) y combinar condiciones.

$sum
Acumulador de suma.
Dónde se usa: en todos los $group con total: { $sum: 1 }.
Por qué: $sum: 1 cuenta documentos; $sum: '$campo' suma valores.

$$NOW
Variable de sistema con la fecha actual del servidor.
Dónde se usa: en CarpetasRepository.moverConDescendientes (updatedAt: '$$NOW').
Por qué: actualizar updatedAt sin traer el documento a Node.

2. Operadores de actualización con pipeline
updateMany con pipeline (array de etapas)
En lugar de { $set: {...} }, se pasa un array de etapas de agregación, que se ejecutan sobre cada documento.
Dónde se usa: en CarpetasRepository.moverConDescendientes.
Por qué: reescribir ancestros de todos los descendientes requiere lógica condicional ($cond, $indexOfArray, $slice) que no se puede hacer con $set simple.

$concatArrays, $cond, $slice, $indexOfArray, $add
Operadores de expresión para manipular arrays y fechas dentro de un pipeline.
Dónde se usa: en moverConDescendientes, para reconstruir la cadena de ancestros de cada descendiente.
Por qué: mover una carpeta exige reescribir los ancestros de todos sus descendientes en una sola operación.

3. Operadores de consulta y actualización (Mongoose)
.where() y .equals()
Encadenan condiciones. consulta.where('campo').equals(valor).
Dónde se usa: en ArchivosRepository.buscar, ActividadesRepository.buscar.
Por qué: construcción condicional del filtro (equivale a { campo: valor } pero permite añadir condiciones dinámicamente).

.regex()
Búsqueda por expresión regular.
Dónde se usa: en ArchivosRepository.buscar (consulta.where('nombre').regex(new RegExp(escaparRegex(filtro.q), 'i'))).
Por qué: búsqueda por fragmento del nombre, case-insensitive.

.sort(), .skip(), .limit()
Ordenación y paginación en find.
Dónde se usa: en los listados (archivos, carpetas, actividades, papelera).
Por qué: orden y paginación en las consultas normales.

.select()
Proyección (incluir/excluir campos).
Dónde se usa: en UsuariosRepository.buscarPorCorreoConPassword (.select('+password')), buscarEnDirectorio (.select('nombre correo')), etc.
Por qué: no traer todo; +campo incluye un campo con select: false.

.lean()
Devuelve objetos JS planos, sin métodos de Mongoose.
Dónde se usa: en CarpetasRepository.nombresDe, idsDescendientesActivas, idsSubarbol, idsEnPapelera, ArchivosRepository.buscarPorCarpetas, ComparticionesRepository.*PorIds, AccesoRepository.carpetaActiva, etc.
Por qué: más rápido y menos memoria cuando solo se va a leer.

.populate()
Rellena un campo ObjectId con el documento referenciado.
Dónde se usa: en ComentariosRepository (autor) y ComparticionesRepository (destinatario, propietario).
Por qué: traer datos del usuario referenciado sin un $lookup; más legible en find.

.collation()
Aplica reglas de idioma (case-insensitive, acentos).
Dónde se usa: en CarpetasRepository.listarPorPadre, existeHermana, ArchivosRepository.buscar, existeHermano, etiquetasDistintas.
Por qué: COLACION_ES ({ locale: 'es', strength: 2 }) hace que "Docs" y "docs" sean iguales.

.countDocuments() y .estimatedDocumentCount()
Cuentan documentos.
Dónde se usa: ArchivosRepository.buscar y ActividadesRepository.buscar para el total paginado.
Por qué: countDocuments cuenta con filtro; estimatedDocumentCount es más rápido pero sin filtro.

.distinct()
Devuelve valores únicos de un campo.
Dónde se usa: en ArchivosRepository.etiquetasDistintas.
Por qué: lista de etiquetas únicas para el filtro.

.exists()
Devuelve true/false si hay al menos un documento.
Dónde se usa: en CarpetasRepository.existeHermana y ArchivosRepository.existeHermano.
Por qué: validar duplicados sin traer el documento.

.findByIdAndUpdate() y .findOneAndUpdate()
Actualizan y devuelven el documento.
Dónde se usa: en CarpetasRepository.renombrar, restaurarRaiz, enviarAPapelera; ArchivosRepository.actualizar, restaurarRaiz, reemplazarContenido, enviarAPapelera; ComparticionesRepository.cambiarNivel; UsuariosRepository.actualizar, ajustarAlmacenamiento.
Por qué: actualizar y devolver en una sola operación.

.updateMany()
Actualiza varios documentos.
Dónde se usa: en CarpetasRepository.moverConDescendientes, enviarDescendientesAPapelera, restaurarDescendientes; ArchivosRepository.enviarAPapeleraPorCarpetas, restaurarPorCarpetas.
Por qué: operaciones en lote (mover carpeta, enviar a papelera, restaurar).

.deleteOne() y .deleteMany()
Borran uno o varios documentos.
Dónde se usa: en ArchivosRepository.eliminarUnoEnPapelera, eliminarPorIds; CarpetasRepository.eliminarPorIds; ComentariosRepository.eliminar, eliminarPorArchivos; ComparticionesRepository.eliminar, eliminarPorRecursos.
Por qué: eliminación definitiva.

.create()
Crea un documento.
Dónde se usa: en todos los crear de los repositorios.
Por qué: inserción con validación del schema.

.exec()
Ejecuta la query y devuelve una Promise.
Dónde se usa: en todas las queries de Mongoose.
Por qué: sin .exec(), Mongoose devuelve una Query "thenable"; con .exec() se obtiene una Promise real y errores mejor tipados.

$set e $inc
Operadores de actualización.
Dónde se usa: $set en casi todas las actualizaciones; $inc en UsuariosRepository.ajustarAlmacenamiento.
Por qué: $inc es atómico y evita race conditions.

4. Índices
Índice simple
Acelera búsquedas por un campo.
Dónde se usa: CarpetaSchema.index({ ancestros: 1 }), ArchivoSchema.index({ etiquetas: 1 }), ComentarioSchema.index({ archivo: 1, createdAt: 1 }), ActividadSchema.index({ fecha: -1 }).
Por qué: acelera las consultas más frecuentes; 1/-1 es el sentido.

Índice compuesto
Índice sobre varios campos, en orden.
Dónde se usa: { propietario, carpetaPadre, nombre }, { propietario, carpeta, nombre }, { propietario, createdAt }, { destinatario, createdAt }, { propietario, createdAt } (en comparticiones), { usuario, fecha }, { recurso, fecha }, { propietarioRecurso, fecha }.
Por qué: las queries filtran por varios campos y ordenan por fecha; el orden importa.

Índice único
Impide duplicados.
Dónde se usa: usuarios.correo, carpetas { propietario, carpetaPadre, nombre }, archivos { propietario, carpeta, nombre }, comparticiones { recurso, destinatario }.
Por qué: reglas de negocio a nivel de base de datos; el código captura el error 11000.

Índice parcial (partialFilterExpression)
El índice solo se aplica a los documentos que cumplen el filtro.
Dónde se usa: carpetas y archivos con { enPapelera: false }.
Por qué: permite tener dos archivos con el mismo nombre si uno está en papelera.

collation en el índice
Aplica reglas de idioma al índice.
Dónde se usa: carpetas y archivos con collation: COLACION_ES.
Por qué: "Docs" y "docs" colisionan; el índice único es case-insensitive.

5. GridFS
GridFSBucket
API de MongoDB para almacenar archivos grandes en dos colecciones (*.files y *.chunks).
Dónde se usa: en ContenidosRepository con bucketName: 'contenidos'.
Por qué: el contenido de los archivos se guarda en GridFS, no en el documento.

openUploadStream() y openDownloadStream()
Streams para subir y descargar.
Dónde se usa: ContenidosRepository.guardar y abrirLectura.
Por qué: no cargar el archivo completo en memoria; se procesa por trozos.

bucket.delete(id)
Borra un archivo de GridFS (documento en files y todos sus chunks).
Dónde se usa: ContenidosRepository.eliminar, invocado desde PapeleraService.eliminarArchivo, eliminarCarpeta y vaciar.
Por qué: al eliminar definitivamente un archivo, se borra también su contenido para no dejar huérfanos.

6. Validación de ObjectId y errores
isObjectIdOrHexString
Valida que un string sea un ObjectId válido.
Dónde se usa: en ParseObjectIdPipe1.
Por qué: los :id de la URL deben ser ObjectIds válidos; si no, se devuelve 400 en vez de 500.
new Types.ObjectId(id)
Convierte un string a ObjectId.
Dónde se usa: en ArchivosRepository.enviarAPapeleraPorCarpetas, restaurarPorCarpetas, CarpetasRepository.moverConDescendientes, ComparticionesService.recursosDe, ContenidosRepository.
Por qué: algunos operadores de MongoDB exigen ObjectId, no string.

error.code === 11000
Detecta violación de índice único.
Dónde se usa: en la utilidad esClaveDuplicada.
Por qué: MongoDB lanza el error 11000 cuando un índice único se viola; se transforma en 409 Conflict.